

Dado um grafo G e um vértice $s \in V(G)$, *quais vértices são alcançáveis a partir de s* ? Os vértices alcançáveis a partir de s são precisamente aqueles que pertencem à mesma componente que s .

3.1 Busca genérica

Seja G um grafo. Uma busca é um algoritmo que visita vértices de grafo G . Em qualquer instante durante a execução de uma busca em G vale que cada vértice $u \in V(G)$ está em um de dois estados possíveis: **não-visitado** ou **visitado**.

O algoritmo abaixo, que chamaremos de $\text{Visite}(G, s)$, recebe um grafo G , um vértice inicial $s \in V(G)$ e *visita todos os vértices alcançáveis a partir de s* .

De início temos duas estruturas de dados L e p vazias, um grafo G dado por matriz ou lista de adjacência e todos os vértices de G estão marcados **não-visitado**.

Para gerenciar as visitas, assumimos que $\text{Visite}(G, s)$ usa um “depósito” L no qual a busca, inserção e remoção de elementos (vértices) é feita de maneira *arbitrária* e com custo fixo. Além disso, o teste “ $L = \emptyset$?” é feito em tempo constante. Usamos vetor p para armazenar o vértice que “descobriu” um novo vértice não-visitado.

Algoritmo 1: $\text{Visite}(G, s)$.

Dado : um grafo G com vértices marcados **não-visitados** e um vértice s .

Devolve : visita todos os vértices de G alcançáveis a partir de s .

```

1 insira  $s$  em  $L$ ;
2  $p[s] \leftarrow \text{nil}$ ;
3 enquanto  $L \neq \emptyset$  faça
4    $u \leftarrow$  remova um elemento de  $L$ ;
5   se  $u$  é não-visitado então
6     marque  $u$  visitado;
7     para cada  $w \in N_G(u)$  não-visitado faça
8        $p[w] \leftarrow u$ ;
9       insira  $w$  em  $L$ ;
```

No que segue faremos uma análise do algoritmo $\text{Visite}(G, s)$ para determinar sua correção e sua complexidade.

Análise do algoritmo.

Fixe um grafo $G = (V, E)$ e um vértice $s \in V(G)$, com todos os vértices inicialmente *não-visitados*. Podemos e **vamos assumir que G é conexo**.

Proposição 3.1: Finitude

O algoritmo Visite (G, s) executa um número finito de iterações do laço **enquanto** da linha 3 e, portanto, termina.

Demonstração. Seja I o número total de inserções em L ao longo da execução. Cada vértice u dispara o laço **para cada** da linha 7 no máximo uma vez, precisamente quando u é removido e encontrado *não-visitado* pela primeira vez (linha 5), instante em que u é marcado *visitado* e permanece assim para sempre. Nesse laço, no máximo $d_G(u)$ inserções ocorrem (uma por vizinho $w \in N_G(u)$ que satisfizer a condição de ser não-visitado; podem ocorrer menos, se algum vizinho já estiver visitado nesse instante, mas não mais). Logo

$$I \leq 1 + \sum_{u \text{ é visitado}} d_G(u) \leq 1 + \sum_{u \in V(G)} d_G(u) = 1 + 2|E|,$$

onde o 1 inicial é a inserção de s na linha 1. Assim I é finito. Como cada iteração do laço (linha 3) remove exatamente um elemento de L (linha 4) e L começa vazio, o número de iterações é igual ao número de remoções, que não pode exceder I ; logo o laço termina após no máximo $I \leq 1 + 2|E|$ iterações com $L = \emptyset$. $\square$

Lema 3.1: Mecanismo de visita

Se um vértice $x \in V(G)$ é inserido em L ao menos uma vez durante a execução de Visite (G, s) , então x é marcado *visitado* em algum momento dessa execução.

Demonstração. Pela Proposição 3.1, o algoritmo termina com $L = \emptyset$; terminar vazio significa que toda ocorrência de todo vértice inserido em L , ao longo da execução, é removido (linha 4) em algum momento.

Em particular se x foi inserido, será removido. Considere a *primeira* dessas remoções de uma ocorrência de x , na ordem temporal de execução.

Como x só pode se tornar *visitado* imediatamente após ser removido e passar pelo teste da linha 5 (não há nenhuma outra instrução no algoritmo que marque um vértice como visitado), no instante dessa primeira remoção x ainda está *não-visitado*. Logo a condição da linha 5 é satisfeita, e x é marcado *visitado* nesse instante. $\square$

Teorema 3.1: Corretude

Ao final de uma execução de *Visite* (G, s), todo vértice $w \in V$ foi marcado *visitado*.

Demonstração. Por indução na distância $\text{dist}_G(s, w)$, mostraremos que todo w com é visitado ao final da execução. De G conexo, $\text{dist}_G(s, w)$ é finita.

Base. $\text{dist}_G(s, w) = 0$, i.e. $w = s$. O vértice s é inserido em L na linha 1; pelo Lema 3.1, s é visitado.

Passo indutivo. Suponha que todo vértice u com $\text{dist}_G(s, u) \leq k$ é visitado em algum momento da execução. Seja w com $\text{dist}_G(s, w) = k + 1$, e tome um caminho

$$s = v_0, v_1, \dots, v_k, v_{k+1} = w$$

que realiza a distância $\text{dist}_G(s, w) = k + 1$.

Então, $v_0, v_1, \dots, v_k$ também é um caminho de comprimento mínimo (justifique), logo $\text{dist}_G(s, v_k) = k$ e, pela hipótese de indução, v_k é visitado em algum instante t da execução.

Há dois casos, dependendo do estado de w no instante t :

1. Se w já estava *visitado* antes de t , então w é visitado, não há mais nada a provar.
2. Se w estava *não-visitado* no instante t : como $\{v_k, w\} \in E(G)$, temos $w \in N_G(v_k)$; no instante t , ao percorrer $N_G(v_k)$, a condição “ w não-visitado” vale, logo w é inserido em L (linha 9). Pelo Lema 3.1, w é visitado em algum momento posterior da execução.

Em ambos os casos w é visitado. Por indução, todo vértice com $\text{dist}_G(s, w)$ finito é visitado ao final da execução de *Visite* (G, s). $\square$

Teorema 3.2: Complexidade do “Visite”

Suponha que a busca, a inserção e a remoção de um elemento de L sejam feitas em tempo $O(T)$, sem nenhuma política específica de gerenciamento, e que o teste e a marcação do estado (*visitado/não-visitado*) de um vértice sejam $O(1)$. Então *Visite* (G, s) tem complexidade de tempo $O(|E| T)$.

Demonstração. Pela Proposição 3.1, o número total de inserções em L ao longo da execução é $I \leq 1 + 2|E| = O(|E|)$, e o número de remoções é $R = I = O(|E|)$ (cada iteração do laço remove exatamente um elemento). Logo o número total de operações sobre L é $O(|E|)$, cada uma custando $O(T)$, o que contribui $O(|E| T)$ ao tempo total, essa parcela cobre as linhas 1, 4 e 9, e também o teste “ $L \neq \emptyset$ ” da linha 3, executado uma vez por iteração.

Resta o custo dos laços **para cada**. Quando u é removido e encontrado *não-visitado* (linha 5), o algoritmo o marca *visitado* — o que só acontece uma única vez para cada vértice, já que um vértice *visitado* nunca retorna a *não-visitado*, e percorre $N_G(u)$: para cada $w \in N_G(u)$, o teste “ w é não-visitado” custa $O(1)$, e quando positivo a atribuição $p[w] \leftarrow u$ custa mais $O(1)$ (a inserção de w em L já foi contabilizada acima, no total de operações sobre L). Logo esse laço custa $O(d_G(u))$, e como cada vértice o dispara no máximo uma vez,

$$\sum_{u \text{ visitado}} O(d_G(u)) \leq \sum_{u \in V(G)} O(d_G(u)) = O(|E|).$$

Somando as parcelas: operações em L + laços **para cada**

$$O(|E| T) + O(|E|) = O(|E| T),$$

para $T \geq 1$ (com a ressalva de que todo modelo razoável de custo por operação tem $T \geq 1$). $\square$

Para visitar os vértices de um grafo G qualquer, não necessariamente conexo, repetimos o `Visite( $G, s$ )` em cada componente conexa:

Algoritmo 2: Busca(G).

Dado : um grafo G .

Devolve : visita aos vértices de G .

- 1 marque todos os elementos de $V(G)$ *não-visitado*;
 - 2 **enquanto** há vértice *não-visitado* **faça**
 - 3 escolha v não-visitado;
 - 4 `Visite( $G, v$ )`.
-

A inicialização tem custo $O(|V(G)|)$. Se $C_1, \dots, C_k$ são as componentes conexas de G , o tempo de cada `visite` é $O(|E(C_i)| T)$ e $\sum_i |E(C_i)| = |E(G)|$, portanto, o tempo total é $O(|V(G)| + |E(G)| T)$.

Teorema 3.3: Complexidade de “Busca”

Suponha que a busca, a inserção e a remoção de um elemento de L sejam feitas em tempo $O(T)$, sem nenhuma política específica de gerenciamento, e que o teste e a marcação do estado (*visitado/não-visitado*) de um vértice sejam $O(1)$. Então `Busca( $G$ )` visita todos os vértices de um grafo (V, E) em tempo $O(|V| + |E| T)$.

3.2 Algoritmos de busca

Há, principalmente, dois casos particulares do algoritmo 1 visto acima. Cada caso é obtido quando definimos uma política de gerenciamento

para L .

3.2.1 Busca em Largura

Se a estrutura de dados L é uma *fila*, com inserção feita no fim da fila e remoção no inícios da fila, então essas instruções, inserções (*push*) e remoções (*pull*), podem ser realizadas em tempo $O(1)$ cada, portanto o algoritmo de Busca em Largura é executado em tempo $O(V + E)$.

3.2.2 Busca em Profundidade

Se a estrutura de dados L é uma *pilha*, com inserção e remoção feitas no topo da pilha, então essas instruções, inserções (*push*) e remoções (*pop*), podem ser realizadas em tempo $O(1)$ cada, portanto o algoritmo de Busca em Largura é executado em tempo $O(V + E)$.

Busca em Profundidade rotulada

A seguinte versão recursiva para a busca em profundidade, que chamamos de *busca em profundidade rotulada*, será usada adiante nesse texto; no início temos $\text{cont} = \text{sai}(v) = \text{chega}(v) = 0$ e $p[v] = \text{nil}$, para todo vértice v de G . A ideia é manter uma variável contadora das operações de inserção e remoção na pilha L . Esse contador é usado para rotular cada vértice com dois valores inteiros: o valor do contador quando o vértice v entrou na pilha, $\text{chega}(v)$, e o valor do contador quando o vértice v saiu da pilha, $\text{sai}(v)$.

Algoritmo 3: BP(G, w).

Dado : um grafo G e um vértice w .

Devolve : busca em profundidade em G a partir de w com rótulos *chega*, *sai* e p em cada vértice.

```

1 cont = 0;
2 cont ← cont + 1;
3 chega(w) ← cont;
4 para cada  $u \in N(w)$  faça
5   se chega(u) = 0 então
6      $p[u] \leftarrow w$ ;
7     BP( $G, u$ );
8 cont ← cont + 1;
9 sai(w) ← cont.
```

Ainda, o algoritmo usa um vetor $p[]$ indexado pelos vértices. No final da execução $p[v]$ tem o vértice que foi empilhado antes de v durante a execução, em outras palavras, foi o vértice que descobriu v .

Proposição 3.2: Rotulação na Busca em Profundidade

Após uma execução de BP (G, w) , para quaisquer $u, v \in V(G)$ alcançáveis a partir de w , um e somente um dos itens abaixo vale para esse par de vértices

1. $\text{chega}(u) < \text{sai}(u) < \text{chega}(v) < \text{sai}(v)$;
2. $\text{chega}(v) < \text{sai}(v) < \text{chega}(u) < \text{sai}(u)$;
3. $\text{chega}(u) < \text{chega}(v) < \text{sai}(v) < \text{sai}(u)$;
4. $\text{chega}(v) < \text{chega}(u) < \text{sai}(u) < \text{sai}(v)$.

Demonstração. A prova é deixada como exercício. $\square$

3.2.3 Distâncias em grafos por busca em largura

Nesta seção adaptamos a busca em largura para calcular, a partir de um vértice fixado s , a distância até todos os demais vértices do grafo. A ideia é simples: em vez de apenas marcar cada vértice como *visitado*, usamos o próprio valor da distância como marcador de estado (um vértice ainda não foi descoberto enquanto $d[\cdot]$ vale ∞) e atribuímos a cada vizinho recém-descoberto a distância do vértice que o descobriu, mais um.

Algoritmo 4: Distância(G, s).

Dado : um grafo G e um vértice $s \in V(G)$.

Devolve : $\text{dist}_G(s, v)$ para todo $v \in V(G)$, e um vetor p de predecessores.

```

1 para cada  $v \in V(G)$  faça  $d[v] \leftarrow \infty$ ;
2  $d[s] \leftarrow 0$ ;
3  $p[s] \leftarrow \text{nil}$ ;
4 insira  $s$  na fila  $L$ ;
5 enquanto  $L \neq \emptyset$  faça
6    $u \leftarrow$  remova da fila  $L$ ;
7   para cada  $v \in N_G(u)$  tal que  $d[v] = \infty$  faça
8      $d[v] \leftarrow d[u] + 1$ ;
9      $p[v] \leftarrow u$ ;
10    insira  $v$  na fila  $L$ ;
```

Fixe um grafo $G = (V, E)$ e um vértice $s \in V(G)$. Para todo $v \in V(G)$, denotamos por $\text{dist}_G(s, v)$ a distância verdadeira entre s e v em G (o número de arestas de um caminho mais curto), com a convenção $\text{dist}_G(s, v) = \infty$ se v não é alcançável a partir de s . Ao longo desta seção, $d[v]$ denota o valor armazenado na variável de mesmo nome pelo algoritmo Distância(G, s) em um dado instante da execução (o contexto deixa claro qual instante).

Usaremos duas observações imediatas sobre o algoritmo. Primeiro, um vértice $v \neq s$ só recebe valores de $d[v]$ e de $p[v]$ no momento em

que é inserido em L , e a condição do laço **para cada** exige $d[v] = \infty$; logo isso acontece no máximo uma vez por vértice, e *depois que* $d[v]$ se torna finito, seu valor nunca mais muda. Segundo, o valor atribuído a v é $d[u] + 1$, em que $u = p[v]$ é o vértice que estava sendo processado.

O primeiro resultado mostra que o vetor de predecessores $p[\]$ sempre codifica um caminho válido até s , cujo comprimento é o valor $d[v]$.

Lema 3.2: Caminho via p

Se, ao final da execução de $\text{Distância}(G, s)$, $d[v] = k$ para algum k finito, então existe uma sequência de vértices *distintos* $v = v_k, v_{k-1}, \dots, v_1, v_0 = s$ tal que $v_{i-1} = p[v_i]$ e $\{v_{i-1}, v_i\} \in E(G)$, para $i = 1, \dots, k$. Em particular, $\text{dist}_G(s, v) \leq d[v]$.

Demonstração. Por indução em $k = d[v]$.

Base ($k = 0$). Então $v = s$: todo vértice diferente de s que recebe valor recebe $d[u] + 1 \geq 1$. A sequência é a trivial $v_0 = s$.

Passo indutivo. Seja $k \geq 1$ e suponha o resultado válido para $k - 1$. Seja $v \neq s$ com $d[v] = k$. O valor de $d[v]$ foi atribuído quando v foi descoberto a partir de $u = p[v]$, com $v \in N_G(u)$ (logo $\{u, v\} \in E(G)$) e $d[u] = k - 1$ (valor que não muda mais).

Por hipótese de indução, existe uma sequência de vértices distintos $u = v_{k-1}, \dots, v_0 = s$ com as propriedades desejadas. Acrescentando $v_k := v$ obtemos $v_k, v_{k-1}, \dots, v_0$.

Essa sequência é um caminho de s a v com $k = d[v]$ arestas. Como $\text{dist}_G(s, v)$ é o comprimento *mínimo* entre os caminhos de s a v , segue $\text{dist}_G(s, v) \leq d[v]$. $\square$

Falta mostrar que os valores $d[v]$ são de fato as distâncias verdadeiras, e é aqui que o fato de L ser uma *fila* (e não uma pilha, nem uma política arbitrária) é essencial. A demonstração usa a estrutura do algoritmo em “camadas”, uma para cada valor de distância.

Para $k \geq 0$, seja $V_k = \{v \in V(G) : \text{dist}_G(s, v) = k\}$. Usaremos dois fatos elementares sobre distâncias:

- (i) se $\{x, y\} \in E(G)$ e $\text{dist}_G(s, x)$ é finito, então $\text{dist}_G(s, y) \leq \text{dist}_G(s, x) + 1$ (um caminho mais curto até x , seguido da aresta $\{x, y\}$, é um passeio de comprimento $\text{dist}_G(s, x) + 1$ até y);
- (ii) todo $v \in V_{k+1}$ tem um vizinho em V_k (o penúltimo vértice de um caminho mais curto de s a v).

Em particular, se $V_k \neq \emptyset$ então $V_j \neq \emptyset$ para todo $j \leq k$.

Lema 3.3: Estrutura em camadas da fila

Para todo $k \geq 0$ tal que $V_k \neq \emptyset$, a execução atinge um instante (o *instante* k) no qual:

- (a) $L = V_k$;

- (b) os vértices v com $d[v] < \infty$ são exatamente os de $V_0 \cup \dots \cup V_k$, e todos eles satisfazem $d[v] = \text{dist}_G(s, v)$.

Demonstração. Por indução em k .

Base ($k = 0$). Tome como instante 0 o momento logo após a linha 4. Nele $L = \{s\} = V_0$ e s é o único vértice com valor finito, $d[s] = 0 = \text{dist}_G(s, s)$.

Passo indutivo. Suponha $V_{k+1} \neq \emptyset$ (o que implica $V_k \neq \emptyset$) e que o instante k existe. Como L é uma fila e $L = V_k$ nesse instante, as próximas $|V_k|$ remoções retiram exatamente os vértices de V_k , e tudo o que é inserido nesse intervalo fica atrás deles. Considere o processamento de um vértice $u \in V_k$ (cujo valor é $d[u] = k$, por (b)) e um vizinho $v \in N_G(u)$. Pelo fato (i), $\text{dist}_G(s, v) \leq k + 1$.

- Se $\text{dist}_G(s, v) \leq k$, então $v \in V_0 \cup \dots \cup V_k$, que por (b) já tem valor finito no instante k ; logo v é ignorado.
- Se $\text{dist}_G(s, v) = k + 1$, então $v \notin V_0 \cup \dots \cup V_k$ e $d[v] = \infty$ no instante k . O primeiro vértice de V_k adjacente a v a ser processado insere v com $d[v] \leftarrow k + 1$; os seguintes o ignoram.

Portanto, durante o processamento de V_k são inseridos apenas vértices de V_{k+1} , cada um uma única vez e com o valor correto $k + 1$; e, pelo fato (ii), todo vértice de V_{k+1} é inserido, pois tem algum vizinho em V_k . Logo, no instante $k + 1$, imediatamente após o último vértice de V_k ser processado, temos $L = V_{k+1}$ e os vértices com valor finito são exatamente os de $V_0 \cup \dots \cup V_{k+1}$, todos com valor correto. $\square$

Combinando os dois lemas, obtemos a corretude do algoritmo.

Teorema 3.4: Corretude

Ao final da execução de Distância(G, s), vale $d[v] = \text{dist}_G(s, v)$ para todo $v \in V(G)$.

Demonstração. Se $\text{dist}_G(s, v) = k$ é finito, então $V_k \neq \emptyset$ e, pelo item (b) do Lema 3.3 no instante k , temos $d[v] = k$; como esse valor nunca mais muda, ele é o valor final.

Se $\text{dist}_G(s, v) = \infty$, suponha por absurdo que $d[v]$ seja finito ao final da execução. Pelo Lema 3.2, existiria um caminho de s a v em G , contradizendo $\text{dist}_G(s, v) = \infty$. Logo $d[v] = \infty$. $\square$

Como consequência imediata, o vetor $p[\]$ não serve apenas para certificar uma distância: ele permite reconstruir explicitamente um caminho mais curto.

Corolário 3.1: Reconstrução de caminho mínimo via p

Para todo $v \in V(G)$ com $\text{dist}_G(s, v)$ finito, a sequência $v = v_k, v_{k-1}, \dots, v_0 = s$ (com $k = \text{dist}_G(s, v)$) obtida seguindo os

ponteiros $v_{i-1} = p[v_i]$ é um caminho *mais curto* de s a v em G .

Demonstração. Pelo Teorema 3.4, $d[v] = \text{dist}_G(s, v) = k$. Pelo Lema 3.2, a sequência é um caminho (de vértices distintos) de s a v com k arestas, que é o menor comprimento possível. Portanto é um caminho mais curto. $\square$

Observação 3.1

O fato de L ser gerenciada como *fila* é usado de forma essencial na demonstração do Lema 3.3: é ele que garante que toda a camada V_k é processada antes de qualquer vértice de V_{k+1} . Com outra política (por exemplo, uma pilha), o algoritmo continua visitando todos os vértices alcançáveis a partir de s , mas $d[v]$ passa a registrar apenas o comprimento de *algum* caminho até v (o encontrado pela política), não necessariamente o mais curto.

Quanto à complexidade, cada vértice é inserido em L no máximo uma vez (a condição $d[v] = \infty$ no laço **para cada** garante isso), e portanto o mesmo argumento usado para o algoritmo genérico $\text{Visite}(G, s)$ se aplica sem modificações: com listas de adjacência e supondo $O(1)$ por operação em L , o algoritmo $\text{Distância}(G, s)$ tem complexidade de tempo $O(|V| + |E|)$.

Exercícios

Exercício 57. Escreva uma versão não recursiva da busca em profundidade rotulada e analise sua complexidade.

Exercício 58. Prove a Proposição 3.2.

Exercício 59. Considere uma execução da busca rotulada $\text{BP}(G, w)$ e defina, para todo $t \in \mathbb{N}$, a t -ésima iteração do vetor p , denotada por $p^{(t)}$, recursivamente, da seguinte maneira

$$p^{(t)}[v] = \begin{cases} v & \text{se } t = 0 \\ p^{(t-1)}[p[v]] & \text{se } t > 0. \end{cases}$$

Prove as seguintes equivalências: para quaisquer $u, v \in V(G)$

1. $[\text{chega}(v), \text{sai}(v)] \cap [\text{chega}(u), \text{sai}(u)] = \emptyset$ se e somente se não existe $t \in \mathbb{N}$ tal que $p^{(t)}[u] = v$ e não existe $t' \in \mathbb{N}$ tal que $p^{(t')}[v] = u$;
2. $[\text{chega}(v), \text{sai}(v)] \subset [\text{chega}(u), \text{sai}(u)]$ se e somente se existe $t \in \mathbb{N}$ tal que $p^{(t)}[v] = u$;
3. $[\text{chega}(v), \text{sai}(v)] \supset [\text{chega}(u), \text{sai}(u)]$ se e somente se existe $t \in \mathbb{N}$ tal que $p^{(t)}[u] = v$.

Exercício 60. Sejam $G = (V, E)$ um grafo e $L \subseteq V$ um conjunto de vértices. O conjunto L define um **corte de arestas** em G , denotado $\nabla_G(L)$ e dado por

$$\nabla_G(L) = \{\{u, v\} \in E : u \in L, v \in \bar{L}\}$$

onde $\bar{L} = V \setminus L$.

Considere o seguinte algoritmo sobre um grafo conexo G

Algoritmo 5: Visite_vértices(G).

Dado : um grafo conexo G .

Devolve : visita aos vértices de G .

```

1  $L, M = \emptyset$ ;
2 marque todos os elementos de  $V(G)$  não-visitado;
3 escolha um vértice  $v$  não-visitado;
4 insira  $v$  em  $L$  e marque  $v$  visitado;
5 enquanto  $\nabla_G(L) \neq \emptyset$  faça
6   escolha  $\{u, w\} \in \nabla(L)$ ;
7   insira  $\{u, w\}$  em  $M$ ;
8    $v \leftarrow$  o extremo de  $\{u, w\}$  em  $\bar{L}$ 
9   insira  $v$  em  $L$ ;
10  marque  $v$  visitado.
```

1. O algoritmo visita todos os vértices de G ? (Pode-se assumir que $\nabla(L) = \emptyset$ se, e só se, $L = V(G)$ – dado que $L \neq \emptyset$ ao longo de toda a execução – e esse fato será provado em um capítulo adiante).
2. Prove que, no final de execução do algoritmo, M contém $|V(G)| - 1$ arestas.
3. Determine a complexidade do algoritmo, propondo uma estrutura de dados que permita testar $\nabla(L) \neq \emptyset$ e escolher uma aresta de $\nabla_G(L)$ eficientemente. Pode-se assumir que as demais operações em L e em M são feitas em tempo constante.
4. Prove que $G[M]$ não contém circuito (dica: indução no número de iterações: cada aresta inserida em M conecta um vértice já em L a um vértice novo (que nunca tocou M antes, pois estava em $\bar{L}$)).

Exercício 61. Um grafo é dito **acíclico** se não contém circuitos. Verifique se o seguinte algoritmo está correto independentemente da política para L :

Algoritmo 6: Acíclico(G).

Dado : um grafo simples G .**Devolve** : *verdadeiro* se G é acíclico, *falso* caso contrário.

```
1  marque todos os vértices de  $G$  não-visitado;
2  para cada  $s \in V(G)$  não-visitado faça
3      insira  $s$  em  $L$ ;
4       $p[s] \leftarrow \text{nil}$ ;
5      marque  $s$  visitado;
6      enquanto  $L \neq \emptyset$  faça
7           $u \leftarrow$  remova um elemento de  $L$ ;
8          para cada  $v \in N_G(u)$  faça
9              se  $v$  é não-visitado então
10                  $p[v] \leftarrow u$ ;
11                 marque  $v$  visitado;
12                 insira  $v$  em  $L$ ;
13             senão
14                 se  $v \neq p[u]$  então retorne falso ;
15  retorne verdadeiro
```
